



GIT IM SDLC

WICHTIGER BAUSTEIN IN DER SOFTWAREENTWICKLUNG

GRUNDLAGEN VON GIT

■ **Was ist Git?**

- Verteiltes Versionskontrollsystem
- Ermöglicht paralleles Arbeiten an Code
- Nachverfolgbarkeit von Änderungen
- Branching und Merging

WICHTIGE BEFEHLE

`git init` – Neues Repository erstellen

`git clone` – Repository kopieren

`git add` – Änderungen zum Staging-Bereich hinzufügen

`git commit -m "Message"` – Änderungen speichern

`git push` – Code ins Remote-Repository hochladen

`git pull` – Neueste Änderungen abrufen

GRUNDLAGEN VON SDLC

1. **Planung** – Anforderungen analysieren
2. **Analyse** – Technische & wirtschaftliche Machbarkeit prüfen
3. **Entwurf** – Architektur und UI/UX planen
4. **Implementierung** – Code schreiben und Git nutzen
5. **Testen** – Fehler finden und beheben
6. **Deployment** – Software ausrollen
7. **Wartung** – Fehler beheben, Updates bereitstellen

GIT IM SDLC

- **Wie hilft Git im Software Development Life Cycle?**
- **Entwurf:** Code-Repositories zur Organisation
- **Implementierung:** Branching für Feature-Entwicklung, sinnvolle Commit-Messages, Code Reviews mit Pull Requests
- **Testen:** Versionierung zur Fehlerbehebung
- **Deployment:** CI/CD-Pipelines mit Git (z.B. automatisiert mit GitHub Actions)

KONKRETES BEISPIEL FÜR CD MIT GITHUB ACTIONS

```
1  name: Deploy to Raspberry Pi 400
2
3  on:
4    pull_request:
5      branches:
6        - main
7
8  jobs:
9    deploy:
10     runs-on: [self-hosted, linux] # Hier wird der selbst gehostete Runner verwendet
11
12     steps:
13
14       # 1. Checkout des Repositories
15       - name: Checkout code
16         uses: actions/checkout@v3
17
18       # 3. Git Pull auf dem Pi400 ausführen
19       - name: Pull latest code on Pi400
20         run: |
21           cd /var/www/html/time-recording && git pull origin main
22
23       # 3. Python-Abhängigkeiten installieren
24       - name: Install Python dependencies
25         run: |
26           cd /var/www/html/time-recording &&
27           source venv/bin/activate &&
28           cd /var/www/html/time-recording/py &&
29           pip install -r requirements.txt
```

- Hier ist ein GitHub-Actions-Workflow zu sehen für das Deployment eines eigenen kleinen Übungsprojekts auf einem Raspberry Pi 400.
- Zuvor wurde ein GitHub Actions Runner auf dem Pi 400 installiert, der mit dem GitHub-Server kommuniziert. Die yaml-Datei wird im Repository in einem Ordner z.B. mit dem Namen .github/workflows gespeichert.
- Wenn auf GitHub ein Pull Request auf den main-Branch erfolgt, wird der Workflow ausgeführt. Die einzelnen Schritte sind im Code als Kommentar aufgeführt.
- Auf GitHub kann man im Repository unter “Actions” logs zu den Workflows einsehen.

FAZIT

- Git erleichtert kollaborative Entwicklung
- Git liefert vor allem an den Stellen Entwicklung, Testing und Deployment wichtige Tools
- Beide Konzepte zusammen steigern Qualität & Effizienz

QUELLEN

- IT-Handbuch für Fachinformatiker*innen. Bonn ¹¹2023, S. 735ff.
- <https://aws.amazon.com/de/what-is/sdlc/> (abgerufen am 12.2.2025)
- <https://github.com/features/actions> (abgerufen am 12.2.2025)



VIELEN DANK

BENEDIKT BRENK